



Aug 12, 2026

Introduction to Deep Learning

From Statistical Learning to Neural Networks

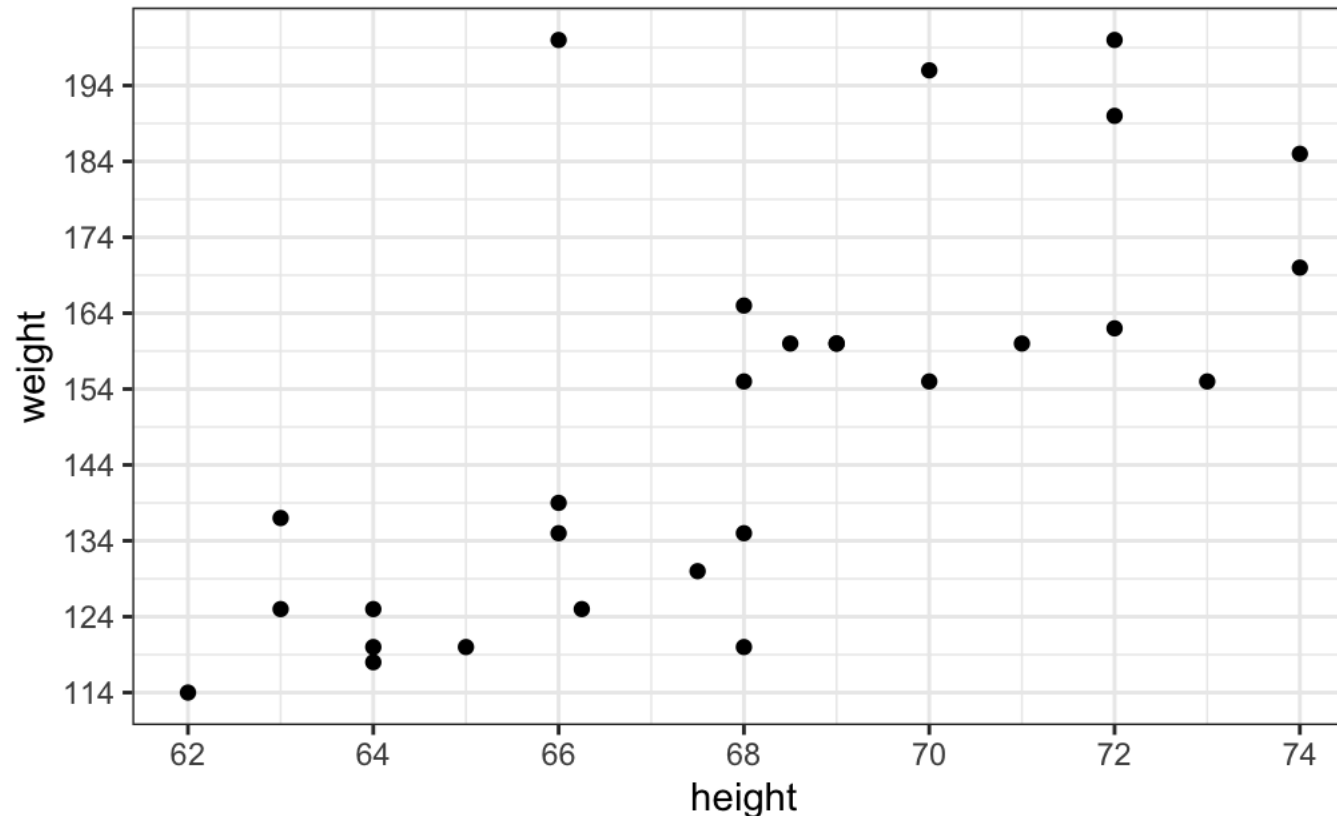
WILBUR OUMA
HPC Partnerships & Outreach Specialist, ALCF

Basics of Statistical Learning

- Basic introduction to machine learning — deep learning!
- Statistical learning, especially regression, can be thought of as a building block of artificial neural networks (many Perceptrons)
- Learning Objectives
 1. Fit a SLR model to data and interpret model coefficients (parameters)
 2. Build a Perceptron to solve a regression problem
 3. Build and train a deep neural network using Keras

Simple Linear Regression: Model Definition

- The goal for SLR is to investigate the relationship between the **response** (Y) and the **predictor** (X) variables
- Recall from high school algebra: $y = b + mx$
 - Where m is the slope and b is the y-intercept



Simple Linear Regression: Model Definition

- The general form of the SLR model closely resembles the algebraic equation ($y = b + mx$):

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- For an individual observation (x_i, y_i), the regression equation becomes:

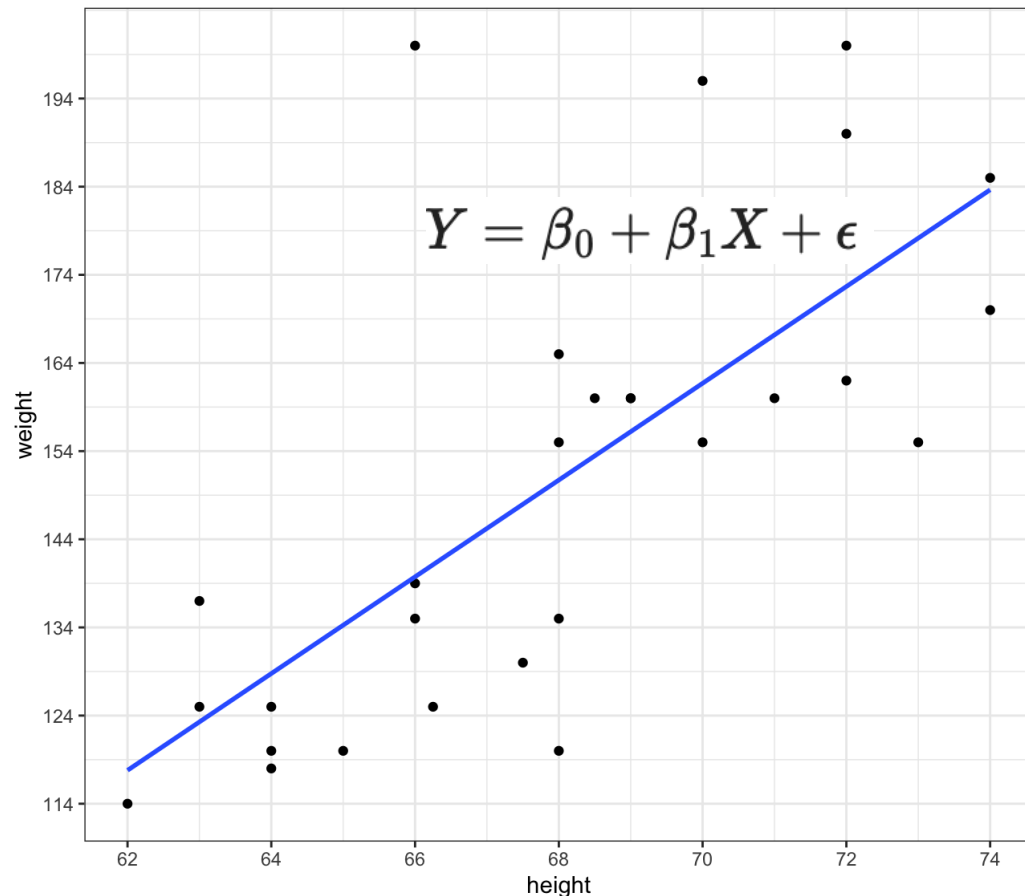
$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

- Where

- β_0 is the population y-intercept,
- β_1 is the population slope
- ϵ_i is the error or deviation of an observation from regression line

- Together, β_0 and β_1 are the (unknown) population model ***coefficients*** or ***parameters***
- The goal of regression is to estimate parameters ($\hat{\beta}$) that describe the relationship between ***Y*** and ***X***
- We estimate $\hat{\beta}$ using the method of Ordinary Least Squares (OLS)

Simple Linear Regression: Errors (Loss)



- Predicted value ($\hat{y}_i$) based on the $\hat{x}_i$ is obtained by

$$\text{fit}_i = \hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

- An error is defined by

$$\text{res}_i = \epsilon_i = y_i - \hat{y}_i$$

- We quantify the total error (loss)

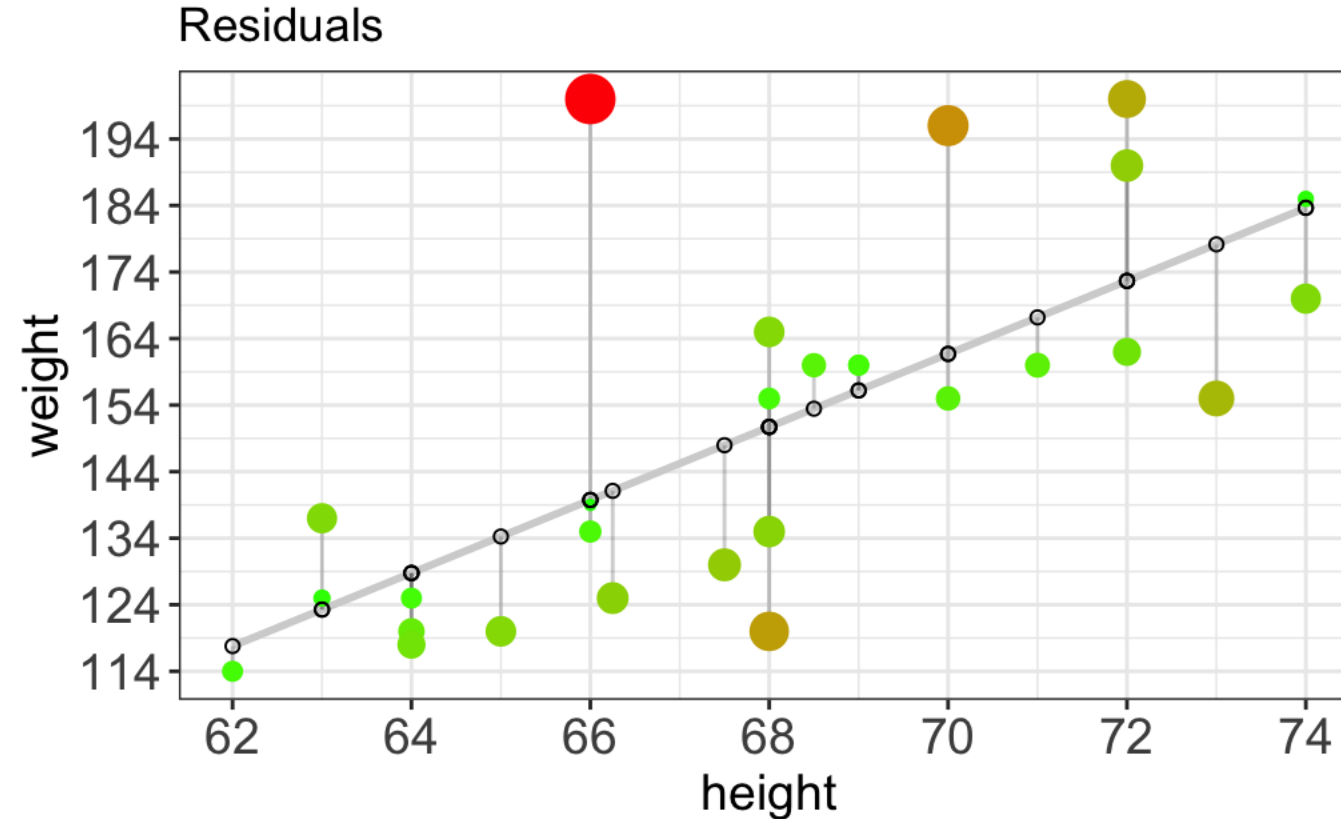
$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

- Goal: obtain least squares estimates of β to minimize MSE

Simple Linear Regression: Errors (Loss)

Errors, visualized



Hands-on: Fitting a SLR model to data



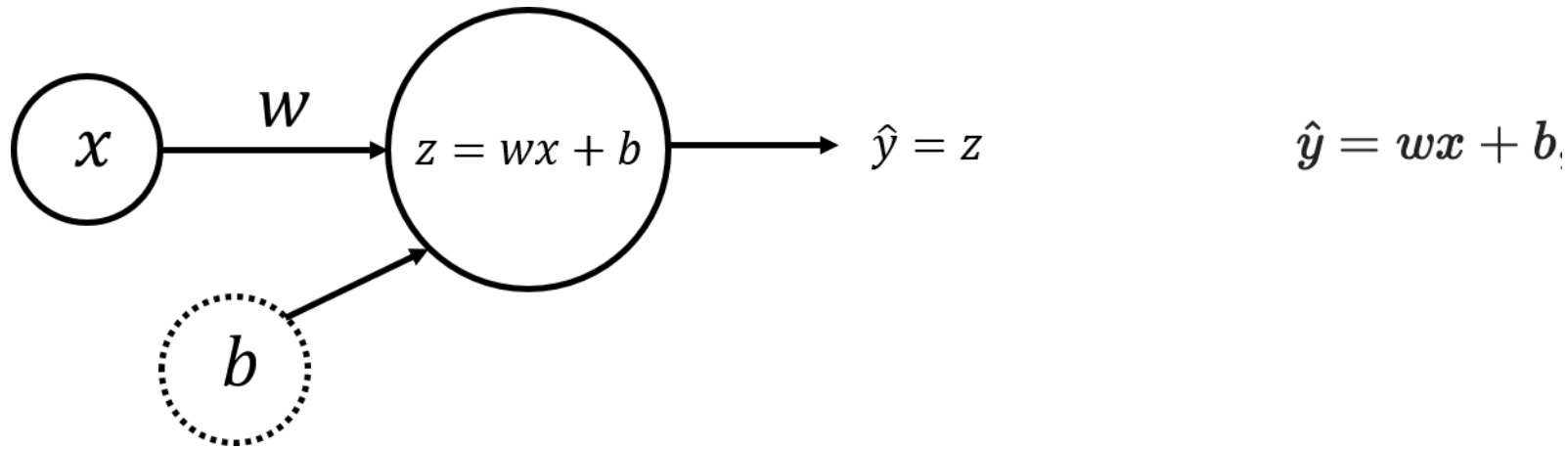
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Regression with Perceptron

- We will construct a 'neuron' corresponding to a SLR model, and train with **gradient descent**



- Weight** (w) and **bias** (b) are the parameters that will get updated when you **train** the model
- As in the previous SLR model, the goal is to identify parameters that minimize the average loss function: **cost function**

$$\mathcal{L}(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Training the model

- After defining the model structure,
 - initialize parameters to some random values (or set to 0) and
 - **update** as the training progresses
- For each training example (x_i, y_i) , predict $\hat{y}_i$:

$$z_i = wx_i + b,$$

$$\hat{y}_i = z_i,$$

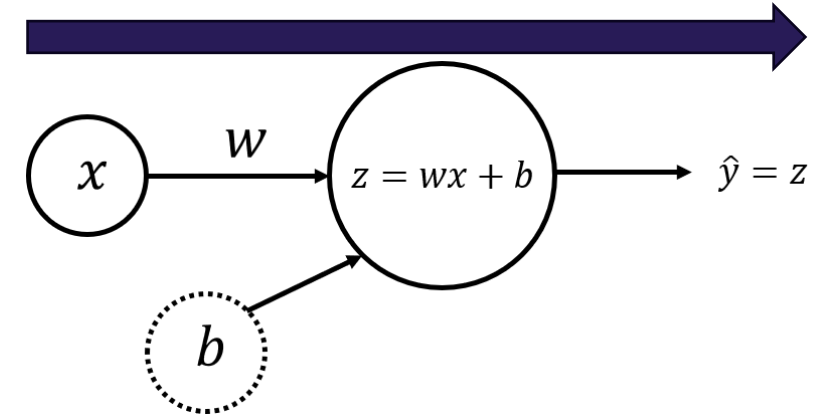
where $i = 1, \dots, m$

- Training examples are a vector X of size $(1 \times m)$
- Perform scalar multiplication of X by a scalar w , adding b .

$$Z = wX + b,$$

$$\hat{Y} = Z,$$

- This set of calculations is called **forward propagation**

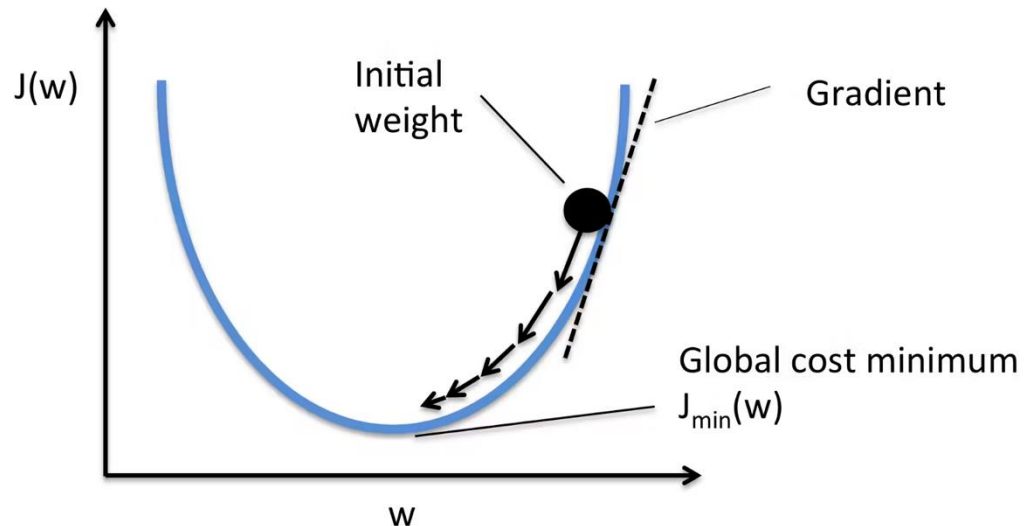


Training the model

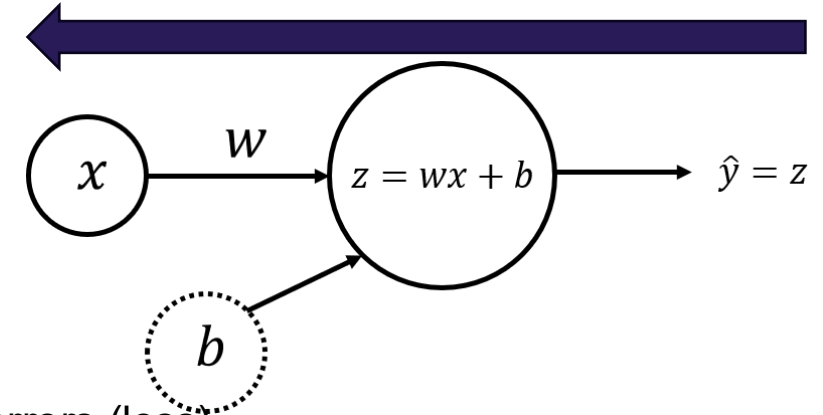
- Given $\hat{Y}$, calculate the cost function

$$\mathcal{L}(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

- Aim is to optimize the cost function during training, hence minimize the errors (loss)
- We minimize the cost function by use of an optimization algorithm, like **gradient descent**
 - Attempt to identify parameter values that minimize the cost function by taking **partial derivatives** of the cost function



- This set of calculations is called **backward propagation**



We calculate partial derivatives as:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_i,$$

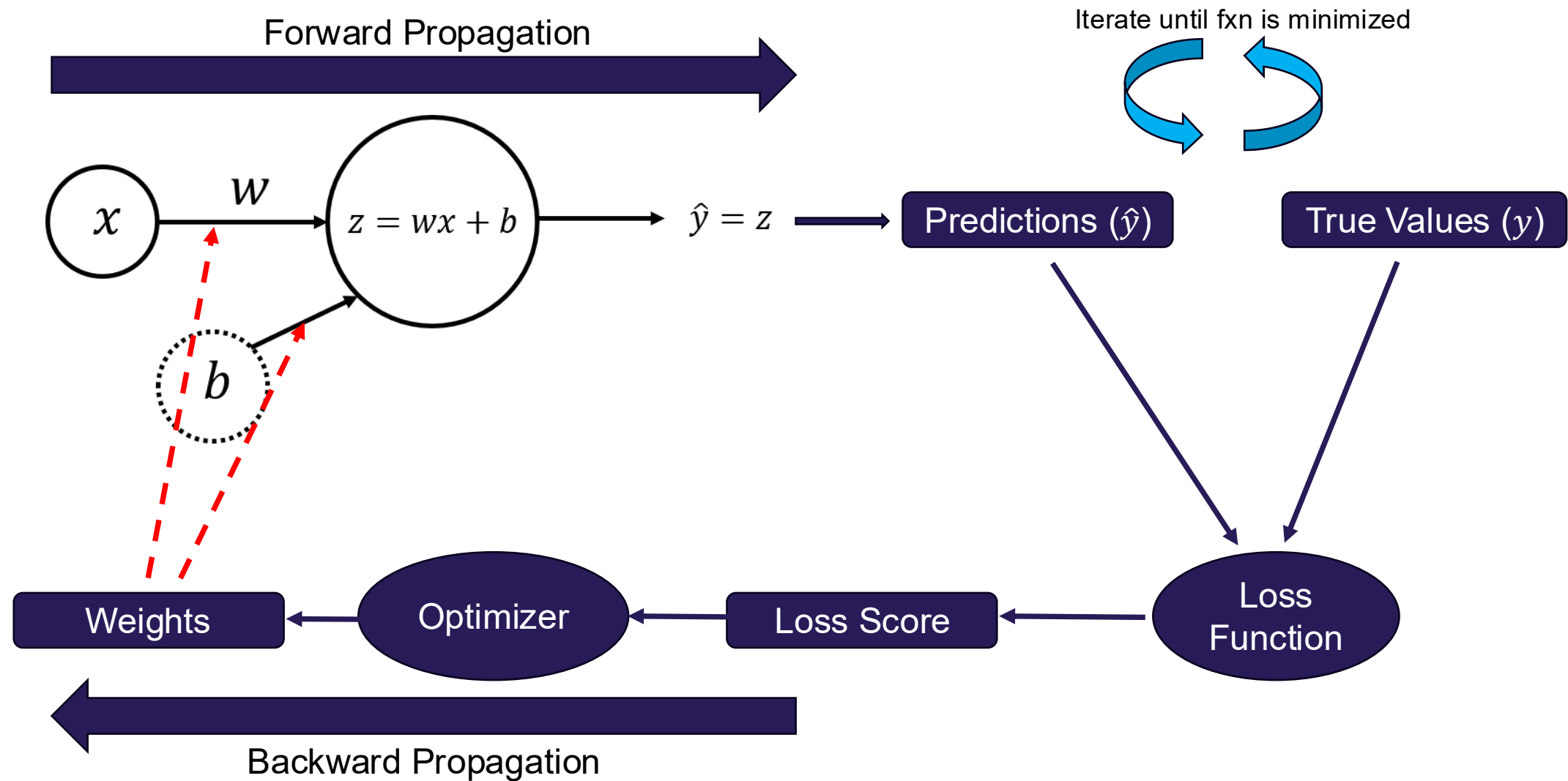
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

We then update the parameters iteratively using the expressions:

$$w = w - \alpha \frac{\partial \mathcal{L}}{\partial w},$$

$$b = b - \alpha \frac{\partial \mathcal{L}}{\partial b}, \quad \text{where } \alpha \text{ is the learning rate}$$

Training the model



Methodology to build a neural network

- Define the neural network structure (# of input units, # of hidden units, etc)
- Initialize the model's parameters
- Loop (iterate)
 - Implement forward propagation (calculate the perceptron output)
 - Implement backward propagation (to get the required corrections for the parameters)
 - Update parameters
- Make predictions with “best” parameters

Hands-on: Regression with Perceptron



U.S. DEPARTMENT OF
ENERGY

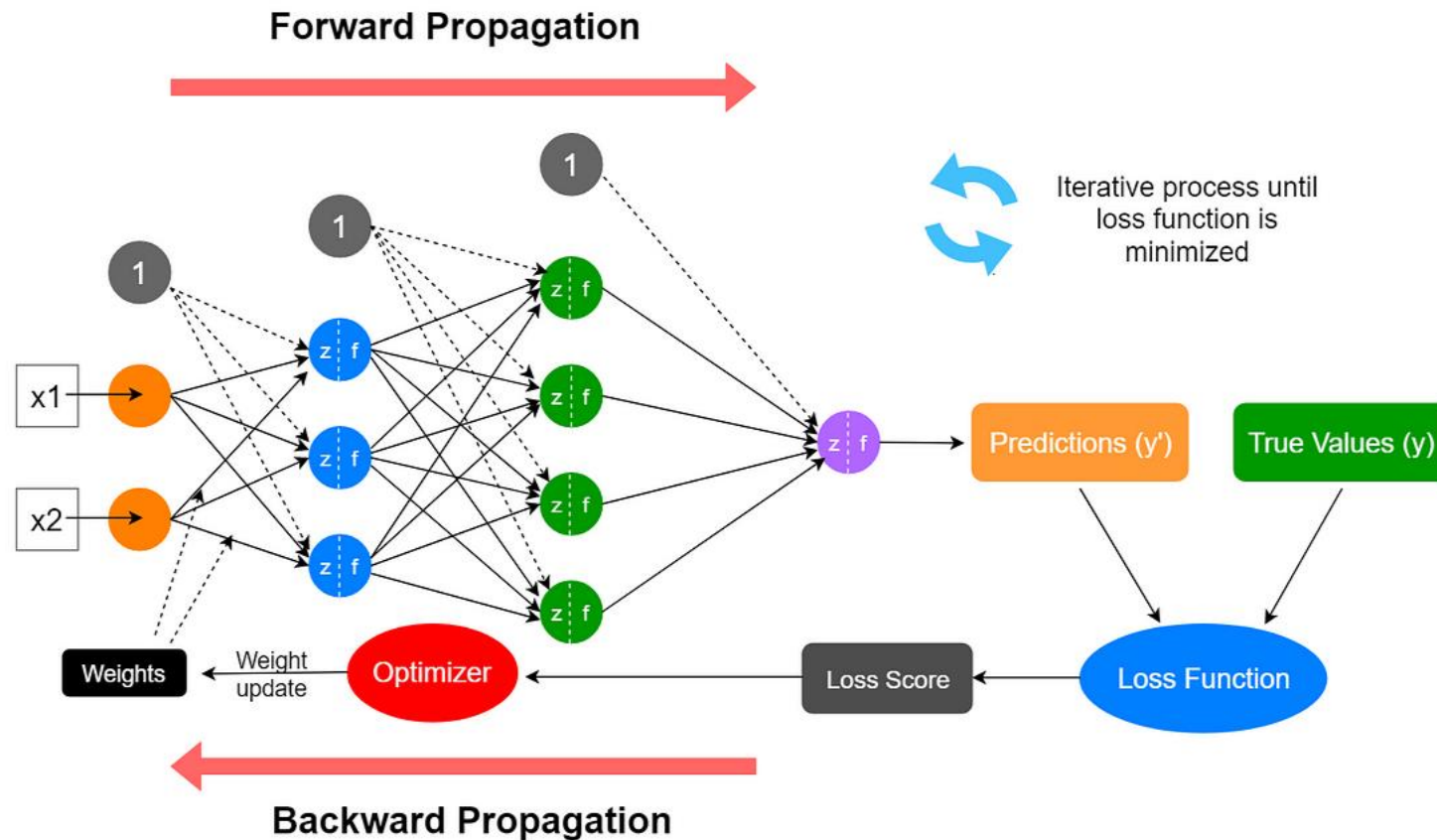
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

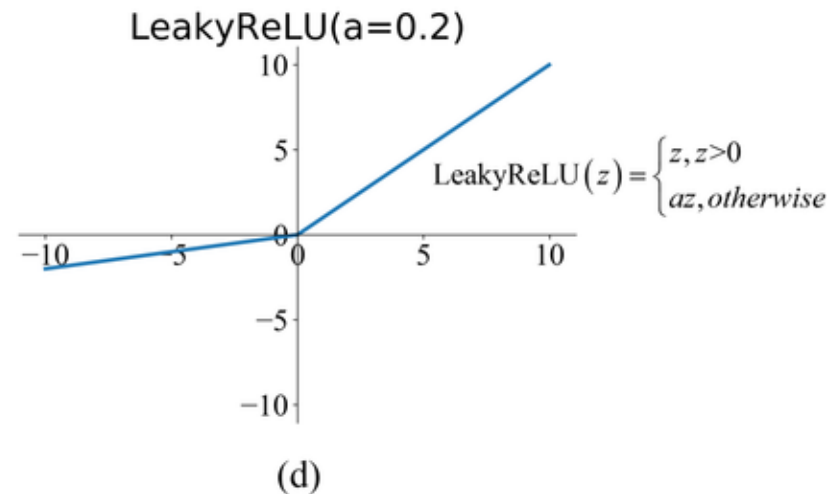
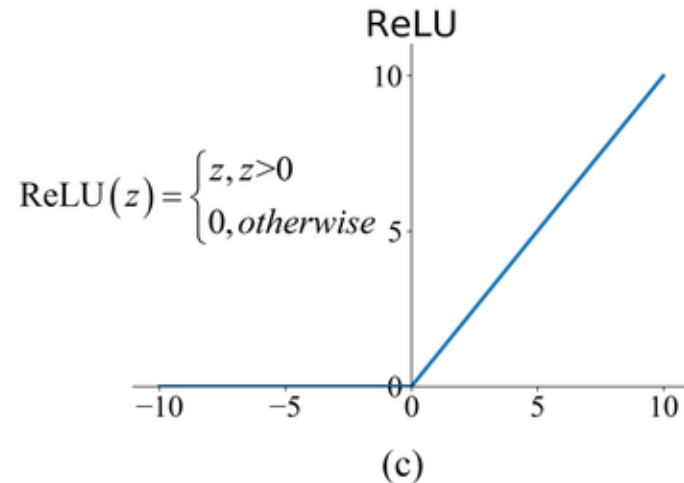
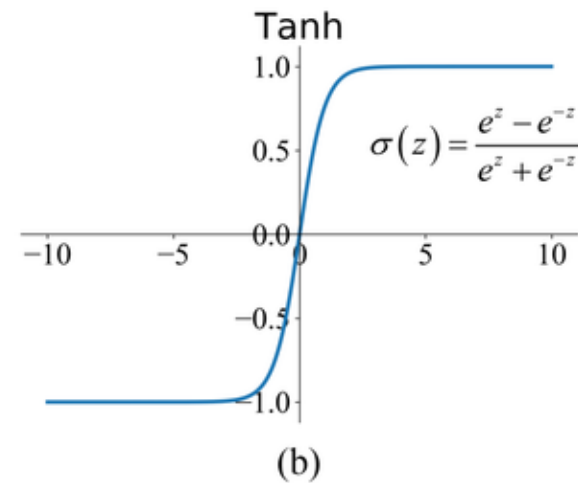
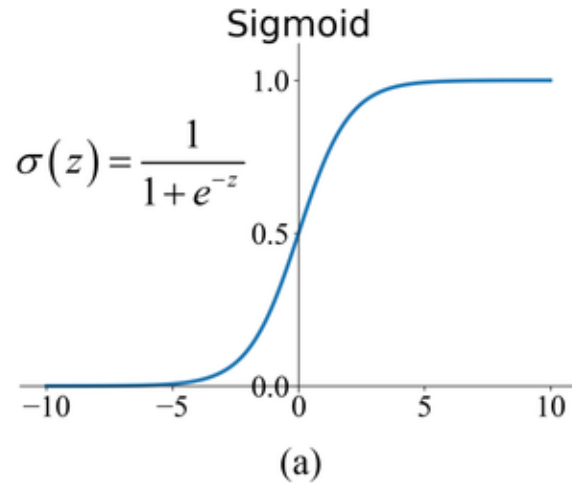
Let's Dive Deep! An Introduction to Deep Learning

The **deep** in deep learning: successive layers of data representation



Let's Dive Deep! An Introduction to Deep Learning

Commonly used activation functions



Hands-on: Building a dense neural net



U.S. DEPARTMENT OF
ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Argonne
NATIONAL LABORATORY



Argonne Leadership
Computing Facility

Next Steps in AI: CNNs, RNNs, and LLMs

As we move beyond simple dense networks, modern AI leverages specialized architectures to tackle complex data types and tasks

Convolutional Neural Networks (CNNs)

- **Purpose:** Designed for **image and spatial data**.
- **Key Idea:** Instead of fully connecting every pixel, CNNs use **convolutional filters** to detect local patterns (edges, textures) and build hierarchical feature maps.
- **Applications:**
 - Image classification (e.g., recognizing objects in photos)
 - Medical imaging
 - Computer vision tasks like segmentation and detection
- **Why important:** CNNs exploit spatial structure, making them far more efficient and accurate for vision tasks than dense networks.

Recurrent Neural Networks (RNNs)

- **Purpose:** Handle **sequential data** where order matters.
- **Key Idea:** RNNs maintain a **hidden state** that carries information across time steps, enabling modeling of temporal dependencies.
- **Variants:**
 - **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)** solve the vanishing gradient problem.
- **Applications:**
 - Natural language processing (NLP)
 - Time-series forecasting
 - Speech recognition
- **Why important:** RNNs introduced the ability to learn from sequences, paving the way for language models.

Large Language Models (LLMs)

- **Purpose:** Power modern AI systems for **text understanding and generation**
- **Key Idea:** Built on **Transformers**, which use **attention mechanisms** to capture relationships between words regardless of position
- **Examples:** GPT, BERT, PaLM
- **Applications:**
 - Chatbots and virtual assistants
 - Code generation
 - Summarization and translation
- **Why important:** LLMs represent a leap in scale and capability, enabling AI systems to perform reasoning, dialogue, and creative tasks.